

# AI Engineer Take-Home Task

## Patient Longitudinal Health Intelligence — Proof of Concept

**Issued:** 20 May 2026, 12:00 PM **Due:** 21 May 2026, 6:00 PM **Duration:** ~7 hours of focused work

### Background

A diagnostic lab has accumulated years of repeat blood test results for thousands of patients. Each patient visits multiple times over months or years. Today, each test report is treated as an isolated event — there is no memory of prior results, no sense of whether a patient is improving or worsening, and no way to know whether two declining markers together signal something more serious than either one alone.

Your task is to build a proof of concept that turns this fragmented data into a coherent health narrative for each patient.

### The Dataset

`patient_health_data.csv` contains de-identified blood test results for **1,000 patients** across **3–23 visits each**, spanning 2019–2026.

| Column                       | Description                                          |
|------------------------------|------------------------------------------------------|
| <code>patient_id</code>      | Anonymous ID (P0001–P1000)                           |
| <code>date_of_test</code>    | Date of the test visit (YYYY-MM-DD)                  |
| <code>param_name</code>      | Machine-readable parameter key                       |
| <code>parameter_group</code> | Organ/system category (glucose, lipid, liver, renal) |
| <code>result</code>          | Numeric test result                                  |
| <code>unit</code>            | Unit of measurement                                  |

| Column                       | Description                                            |
|------------------------------|--------------------------------------------------------|
| <code>low_range</code>       | Lower bound of the normal reference range              |
| <code>high_range</code>      | Upper bound of the normal reference range              |
| <code>parameter_label</code> | Human-readable parameter name                          |
| <code>gender</code>          | Patient gender (Male/Female)                           |
| <code>visits</code>          | Total number of visits this patient has in the dataset |
| <code>age_band</code>        | Age bracket in 5-year bands (e.g. 45-49)               |

### 11 parameters across 4 organ systems:

| Parameter                    | Label           | Group   | Normal Range  |
|------------------------------|-----------------|---------|---------------|
| <code>creatinine</code>      | Creatinine      | Renal   | 0.6–1.2 mg/dL |
| <code>uric_acid</code>       | Uric Acid       | Renal   | 3.4–7.0 mg/dL |
| <code>fasting_glucose</code> | Fasting Glucose | Glucose | 70–109 mg/dL  |
| <code>hba1c</code>           | HbA1c           | Glucose | 0–5.6%        |
| <code>ldl</code>             | LDL             | Lipid   | 0–100 mg/dL   |
| <code>hdl</code>             | HDL             | Lipid   | 40–60 mg/dL   |
| <code>triglycerides</code>   | Triglycerides   | Lipid   | 0–150 mg/dL   |
| <code>sgot</code>            | SGOT (AST)      | Liver   | 0–40 U/L      |
| <code>sgpt</code>            | SGPT (ALT)      | Liver   | 0–41 U/L      |
| <code>total_bilirubin</code> | Total Bilirubin | Liver   | 0–1.0 mg/dL   |
| <code>total_protein</code>   | Total Protein   | Liver   | 6.6–8.2 g/dL  |

Note: `total_protein` is one of the parameters where a result *below* the low range is the concern. `hdl` is protective — low HDL is the risk signal, not high. Factor this into your trend direction logic.

## What You Need to Build

### Deliverable 1 — Jupyter Notebook ( `analysis.ipynb` )

A well-structured, end-to-end notebook with the following sections:

## Section 1: Data Loading and Exploration

- Load and inspect the dataset
- Assess data quality: missing values, coverage per parameter, visits per patient
- Visualise the distribution of visits, parameters, and result ranges

## Section 2: Longitudinal Trend Signals

For each **patient** × **parameter** combination, compute: - Direction of change (improving / worsening / stable) - Magnitude of change (first to last, total range) - Rate of change (slope via linear regression or equivalent) - Whether the patient has crossed the normal boundary (normal→abnormal, or abnormal→normal) - Whether the patient is currently within or outside the normal range - Number of visits where the result was abnormal

Use these signals to assign each patient–parameter pair a **trajectory label** from this set:

| Label                                    | Meaning                                                      |
|------------------------------------------|--------------------------------------------------------------|
| <code>steadily_worsening</code>          | Consistent deterioration, multiple visits                    |
| <code>approaching_risk</code>            | Currently normal but measurably drifting toward the boundary |
| <code>new_problem_emerging</code>        | Was normal in earlier visits, abnormal in recent ones        |
| <code>chronically_abnormal_stable</code> | Persistently abnormal but not getting worse                  |
| <code>relapsing</code>                   | Was normal/improved, now abnormal again                      |
| <code>recovering</code>                  | Was abnormal, now improving                                  |
| <code>steadily_improving</code>          | Consistent improvement across multiple visits                |
| <code>stable_normal</code>               | Within normal range with no meaningful trend                 |
| <code>insufficient_data</code>           | Too few visits to classify reliably                          |

Document the classification logic clearly in the notebook.

## Section 3: Cross-Parameter Burden Analysis

- For each patient, compute how many parameters are currently abnormal
- Compute a simple health burden score at each visit (e.g. number of abnormal params, or sum of normalised deviations)

- Track burden score over time to see whether a patient's overall health is improving or deteriorating
- Identify which pairs of parameters most frequently appear abnormal together across the patient population (co-occurrence analysis)
- For co-occurring pairs, determine which parameter tends to become abnormal *first* (temporal ordering)

### Section 4: LLM-Generated Health Summaries

Using an LLM of your choice (Claude, GPT-4, Gemini, or any open model), generate a short plain-language health summary for each patient based on: - Their trajectory labels across all parameters - Their burden score trend - Their most recent results vs normal range

The summary should be 2–4 sentences, written as if for a clinical care coordinator — not for the patient directly. It should highlight what is worsening, what is stable, and whether anything needs urgent attention.

You are free to design the prompt however you like. Document your prompt design choices.

### Section 5: Findings

- What proportion of the 200 patients are in each trajectory category for their worst-performing parameter?
- Which parameters most commonly drive urgent classifications?
- Which 3+ parameter pairs show the strongest co-occurrence?
- Which 3+ parameter pairs show a consistent temporal ordering (one becomes abnormal before the other)?

## Deliverable 2 — Output CSV ( `patient_summary.csv` )

One row per patient with the following columns:

| Column                    | Description                    |
|---------------------------|--------------------------------|
| <code>patient_id</code>   | Patient identifier             |
| <code>gender</code>       | Gender                         |
| <code>age_band</code>     | Age band                       |
| <code>total_visits</code> | Number of distinct test visits |
| <code>param_trends</code> |                                |

| Column                               | Description                                                                                                                                |
|--------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
|                                      | JSON dict: <code>{param_name: trajectory_label}</code> for all parameters this patient has data for                                        |
| <code>current_abnormal_params</code> | Comma-separated list of parameters currently outside normal range                                                                          |
| <code>burden_trend</code>            | Direction of overall health burden: <code>improving</code> , <code>worsening</code> , <code>stable</code> , <code>insufficient_data</code> |
| <code>worst_trajectory</code>        | The most urgent trajectory label across all parameters                                                                                     |
| <code>llm_summary</code>             | The LLM-generated plain-language health summary                                                                                            |

Example `param_trends` value:

```
{"creatinine": "stable_normal", "ldl": "steadily_worsening", "hba1c": "approaching_risk",
```

## Scope

**In scope:** - All analysis in the Jupyter notebook - The output CSV - Any visualisations that help communicate findings - LLM integration for summaries (API call, local model, or any accessible service)

**Out of scope:** - Any live patient data or external systems - Clinical validation or medical advice - Production-grade code quality (this is a POC)

**Bonus (not required, but valued):** - A simple Streamlit or Gradio dashboard that lets you browse individual patients, see their trend charts per parameter, and read their LLM summary - A patient risk ranking — a sorted table of all 200 patients from most to least urgent based on their trajectory and burden signals

## Evaluation Criteria

| Area                       | What we look for                                                                                          |
|----------------------------|-----------------------------------------------------------------------------------------------------------|
| <b>Analytical thinking</b> | Is the trajectory classification logic sound? Are edge cases handled or at least acknowledged?            |
| <b>Code quality</b>        | Is the notebook readable, reproducible, and well-structured? Can we follow the logic without explanation? |

**LLM integration**

| Area          | What we look for                                                                          |
|---------------|-------------------------------------------------------------------------------------------|
|               | Is the prompt well-designed? Do the summaries reflect the actual data for each patient?   |
| Communication | Do the findings section and summaries communicate clearly to a non-technical reader?      |
| Curiosity     | Did you explore the data and find something interesting that wasn't explicitly asked for? |

## Submission

Please share: 1. `analysis.ipynb` — the executed notebook (with all outputs visible)  
 2. `patient_summary.csv` — the output CSV  
 3. Any additional files (Streamlit app, charts, etc.) you want to include

Send via email or a shared folder link.

## Notes

- The dataset spans 2019–2026; some patients have only 2–3 visits, others have 15+. Your trajectory classification should handle sparse data gracefully and use the `insufficient_data` label where appropriate.
- Reference ranges in the dataset are population-level defaults. For this POC, treat them as the ground truth for normal vs. abnormal.
- Not every patient has data for all 11 parameters — coverage varies.
- You are free to use any Python libraries, LLM APIs, or tools you prefer.
- If you make assumptions or take shortcuts, document them — that is more useful to us than hidden decisions.